

Kernel Hacking 101

- Questa presentazione copre due diversi corsi. In particolare, copre
 - Tutto il modulo di kernel hacking del corso di Sistemi Operativi, per gli studenti della triennale
 - L'introduzione/riepilogo degli elementi di base per il corso di kernel hacking per gli studenti della magistrale



Partiamo!

- Per rendere le cose più divertenti, iniziamo alla cieca ...
 - Scarichiamo, compiliamo, installiamo ed eseguiamo il nostro primo kernel *custom*
 - Trovate tutte le istruzioni in *1-compilazione*
 - Anche se ancora non conosciamo né gli internal del kernel né il sistema di *build*
- Nel frattempo, seguiamo un po' con questo corso ogni volta che dobbiamo aspettare un nuovo passo bloccante in *1-compilazione*
- Man mano che si costruisce il kernel, diventiamo più competenti su quello che stiamo facendo

Cos'è il kernel hacking?

- Modificare kernel per migliorarli
 - Correggere errori
 - Estendere il supporto hardware
 - Aggiungere funzionalità
 - Aumentare la sicurezza
 - Aumentare le prestazioni
 - Migliorare l'interfaccia utente
 - Migliorare la qualità del progetto e del codice
 - Procurandosi informazioni e conoscenze in autonomia
- 3

Conoscenze che dovete già avere

- Compilazione di un progetto software
- Programmazione nei linguaggi in cui è scritto il kernel
 - C, Assembly, minilinguaggio di make
- Collaudo di un programma
 - Black box
 - White box

Distribuzione Linux

- Che cosa sta girando sulla nostra macchina fisica, oppure nella nostra macchina virtuale?
- Una cosiddetta distribuzione Linux:
https://en.wikipedia.org/wiki/Linux_distribution
- Linux è solo il kernel di questo sistema operativo

Dalla teoria dei SO a un kernel reale

- Il kernel Linux è un progetto che cambia così rapidamente, che è impossibile scrivere un libro di testo tecnico, che rimanga completo e corretto per più di qualche mese
- E' impossibile anche disporre di, o divenire, un esperto in grado di rispondere a qualsiasi domanda
- La conoscenza è diffusa
 - In tanti documenti (di ogni tipo), molti sul WEB
 - Nelle menti degli esperti
 - Nel codice stesso
 - *“Il codice è la documentazione”*

Risorse online

- Paradossalmente, l'informazione di buona qualità sul WEB
 - pagine WEB, forum, mailing list, ...

può restare valida ed aggiornata più a lungo di quella contenuta in un libro di testo
- Perché viene tenuta sotto controllo dal feedback degli utilizzatori
- Ma gli utilizzatori non sono necessariamente delle autorità del settore
 - Quindi la correttezza non è assicurata
- Uno dei problemi principali è l'obsolescenza

Ricerca e uso delle fonti

- Bisogna imparare a muoversi in questo contesto, ed a trovare informazioni attendibili
 - Spesso cercando e confrontando più di una fonte sullo stesso argomento
- Nel frattempo bisogna accrescere le proprie conoscenze, il che fa migliorare anche nelle ricerche successive
- In questo corso faremo ricorso spesso a risorse WEB
 - Cercando di aggiornare i riferimenti quando non più validi

Inconvenienti

- Dato un qualche problema di kernel hacking su cui ci si trova a lavorare, la difficoltà non è data solo dalla possibilità che non esista un manuale in cui trovare la soluzione già pronta
- Ma dal fatto che nel mondo del kernel hacking le leggi di Murphy valgono con particolare vigore
- Gli inconvenienti sono pane quotidiano
- Perché i kernel
 - sono sistemi complessi
 - non girano all'interno di uno spazio utente condiviso, che sopravvive ai *crash* e che fornisce strumenti di debugging semplici ed efficaci
 - Non c'è quindi un ambiente a monte, in cui svolgere con estrema facilità l'attività di debugging, tracing e quant'altro

In merito all'ultimo problema

- Le sequenze di azioni, ed i relativi risultati proposti in questo corso valgono nel caso di zero inconvenienti
 - Ma può capitarvi qualche inconveniente
 - Per quanto il percorso sia costruito per essere più stabile possibile
- Però proprio nel caso di inconvenienti farete i passi più importanti nella acquisizione di competenze da kernel hacker
 - Trovare (con l'aiuto del docente) soluzioni a problemi inattesi

Livello di difficoltà scelto

- Il più basso possibile, in conseguenza a quanto detto prima
- Per qualsiasi argomento
 - vedremo solo gli strumenti e le modalità più semplici
 - svolgeremo esercizi semplici e minimali
- Motivazione
 1. Toccare con mano ed imparare i meccanismi di base, su cui poggiano tutti gli strati di successivi
 2. Aggiungere il livello minimo possibile di difficoltà, alla già alta difficoltà del contesto

Compilazione del kernel ...

- Come procede la compilazione?
- Se non siete già arrivati in fondo alla installazione del kernel, ricordatevi di procedere con i vari passi in *1-compilazione*
- E di proseguire con lo studio di queste slide solo mentre non avete nulla da fare per la compilazione del kernel

Indice

- Facciamo visita al kernel ed alla sua documentazione online
- Dove ed in che forma vive il nostro kernel running, all'interno del filesystem
- Il processo di avvio
- Elixir e la nostra prima visita al codice sorgente: il codice di avvio del kernel
- I nostri primi due hack del kernel!

Andiamo a trovare Linux

- Partiamo dal visitare la casa ufficiale:
<https://www.kernel.org>
- Una delle prime cose che si notano è che esistono tante versioni del kernel. Analizziamo il relativo processo di sviluppo, nella sezione 2.1 della seguente documentazione:
<https://docs.kernel.org/process/2.Process.html#the-big-picture>
- Poi esploriamo tutti i contenuti seguendo tutti i link presentati in kernel.org

Documentazione

- Uno dei link porta alla documentazione ufficiale
 - <https://docs.kernel.org/>
- La prima cosa che noterete è che la documentazione è tutto tranne un manuale organico che vi guidi nell'apprendimento
- Altro grave problema è che persino alcune di queste pagine capita che contengano informazioni obsolete (ed errate)
 - Ricordate? Imparare a cavarsela ...
- Leggiamo la pagina principale e facciamoci un'idea. Apriamo le pagine che ci incuriosiscono maggiormente. Per ora è normale che facciamo fatica a seguire questi contenuti

Indice

- Facciamo visita al kernel ed alla sua documentazione online
- Dove ed in che forma vive il nostro kernel running, all'interno del filesystem
- Il processo di avvio
- Elixir e la nostra prima visita al codice sorgente: il codice di avvio del kernel
- I nostri primi due hack del kernel!

Due forme

- L'albero dei sorgenti
- Il kernel eseguibile installato

L'albero dei sorgenti

- Guardiamo assieme la sezione *Linux source code layout* in:
<https://linux-kernel-labs.github.io/refs/heads/master/lectures/intro.html>
- In particolare, vi troviamo una breve descrizione del contenuto di ciascuna sottodirectory

La directory /boot

```
cd /boot
```

```
ls
```

- Elementi principali (concettualmente), per ciascuna istanza di kernel installata:

Immagine [compressa] del kernel

`vmlinu[x|z]-<versione>`

Archivio *ramfs* di avvio (*initramfs*)

`initrd-<versione>`

Configurazione del kernel

`config-<versione>`

- Impareremo cosa è ciascun elemento. Partiamo dall'immagine del kernel.
- Per cominciare, quale kernel è quello in esecuzione?
- Come lo scopriamo?

Versione kernel in esecuzione

```
uname -a
```

Immagine del kernel

- <https://en.wikipedia.org/wiki/Vmlinux>
- vmlinuz è in forma compressa, il cui formato è *bzImage*. A volte si trova tale file immagine proprio col nome *bzImage*
`file vmlinuz`
- Per approfondire l'immagine del kernel, andiamo a vedere direttamente cosa fa, partendo dall'avvio del kernel
- Torneremo poi sull'archivio dell'initramfs, e sul config

Indice

- Facciamo visita al kernel ed alla sua documentazione online
- Dove ed in che forma vive il nostro kernel running, all'interno del filesystem
- Il processo di avvio
- Elixir e la nostra prima visita al codice sorgente: il codice di avvio del kernel
- I nostri primi due hack del kernel!

Processo di avvio

- <https://web.archive.org/web/20071011074709/http://www.ibm.com/developerworks/library/l-linuxboot/index.html>

Indice

- Facciamo visita al kernel ed alla sua documentazione online
- Dove ed in che forma vive il nostro kernel running, all'interno del filesystem
- Il processo di avvio
- Elixir e la nostra prima visita al codice sorgente: il codice di avvio del kernel
- I nostri primi due hack del kernel!

Elixir

- <https://elixir.bootlin.com/linux/>
 - Navigazione ipertestuale del codice del kernel Linux
 - Tra tutte le versioni esistenti
- Ricerca di simboli
 - Documentazione, definizione/definizioni e riferimenti (punti in cui il simbolo è utilizzato)
 - Ci possono essere più definizioni di uno stesso simbolo, per esempio perché dipendenti dall'architettura

Indice

- Facciamo visita al kernel ed alla sua documentazione online
- Dove ed in che forma vive il nostro kernel running, all'interno del filesystem
- Il processo di avvio
- Elixir e la nostra prima visita al codice sorgente: il codice di avvio del kernel
- I nostri primi due hack del kernel!

Codice di avvio del kernel

- Diamo uno sguardo ad *head.S*
- Esatta definizione dipende dall'architettura
- Per esempio, analizziamo
https://elixir.bootlin.com/linux/v6.16.7/source/arch/x86/boot/compressed/head_64.S
- Invoca la `extract_kernel`, una cui implementazione è per esempio in `arch/x86/boot/compressed/misc.c`
 - Questa è la funzione che decompime l'immagine del kernel

Approfondimento processo di avvio

- Studiamo i passaggi principali più in dettaglio, in <https://opensource.com/article/18/1/analyzing-linux-boot-process>
- Ora cerchiamo ed analizziamo la `start_kernel` su *lxr*
 - Ci interessa quella definita in `init/main.c`
 - Contiene:
 - Una lunga sequenza di funzioni di inizializzazione, una per ogni struttura dati principale o componente del kernel
 - Seguite da una generica `rest_init`
- Analizziamo il seguente aspetto importante, poi torniamo sulla `rest_init`

Approfondimento processo di avvio

- Sia alcune funzioni invocate dalla `start_main`, che la `start_main` stessa, contengono invocazioni a funzioni nella forma *pr_<livello>*
- Ad esempio, notiamo una
`pr_notice("%s", linux_banner);`
 - Evidentemente prova a scrivere un messaggio informativo
 - Durante il boot
- Vediamo però dei messaggi durante il boot?

Avvio *quiet*

- Quasi certamente no
- Perché il kernel in una distribuzione si avvia tipicamente in modalità *quiet*
- Per inciso, durante il boot appare anche il cosiddetto *splash screen*, perché il kernel è avviato tipicamente anche con l'opzione *splash*
- Andiamo a scoprire come cambiare questa configurazione. A tale scopo, impariamo
 - 1) Che il kernel viene invocato con una riga di comando
 - 2) Come modificare i parametri su tale riga di comando

Parametri di avvio

- Il kernel linux viene avviato passando dei parametri, con la stessa sintassi di una classica riga di comando:
<https://docs.kernel.org/admin-guide/kernel-parameters.html>
- Esercizio facilissimo: individuare i parametri che fanno sì che il kernel parta in modalità *quiet* e *splash*
- Chi passa questi parametri all'avvio?

Configurazione GRUB

- Ovviamente il bootloader. Ormai GRUB è lo standard.
- Nel file di configurazione di GRUB troviamo proprio i parametri passati sulla command line di avvio del kernel:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash console=tty0"
```

- Ecco una guida a tale file di configurazione:
https://help.ubuntu.com/community/Grub2/Setup#Configuring_GRUB_2
- **Esercizio**
Far sì che GRUB, invece di avviare subito un kernel, si blocchi e proponga un menù, aspettando la scelta dell'utente per un tempo infinito

Soluzione

- Modificare */etc/default/grub* affinché contenga le righe
GRUB_TIMEOUT_STYLE=MENU
GRUB_TIMEOUT=-1
- Eseguire
sudo update-grub2
- Riavviare
- All'avvio dovrebbe apparire un menù

Modalità menù di GRUB

- Ogni entry del menù corrisponde ad un insieme di righe di comando da eseguire
- Premendo invio sulla entry correntemente selezionata, si avvia l'esecuzione di tali righe di comando
- Si può visualizzare e modificare una entry, premendo il tasto **e**
 - Se poi si preme *Ctrl-X*, le righe di comando nella entry vengono eseguite, con le eventuali modifiche
- In ogni caso, le modifiche non sono permanenti, e si annullano dopo l'uscita dalla modalità di modifica

Utilizzo della modalità menù

- Il tipo principale di entry è quella di avvio di una istanza di un sistema operativo
- Ci possono essere più entry di avvio di sistemi operativi, per esempio se ci sono più versioni o in generale varianti di kernel installate
- Ci sono poi entry che rappresentano ulteriori sotto-menù da aprire
 - Ad esempio, dopo aver installato dei kernel custom, questi possono non apparire nel menu principale, ma all'interno di qualche sotto menù (es.: `Advanced options for Ubuntu`)
- Andiamo in edit su una entry che avvia un sistema operativo ed analizziamo solo il seguente comando principale

Comando di avvio di un kernel/SO

- `linux`
 - Questo è il comando che avvia un kernel
 - Sulla riga di comando sarà senz'altro presente il nome dell'immagine del kernel
 - Ci saranno poi i parametri da passare al kernel
 - Un parametro fondamentale è l'identificatore della partizione di dispositivo di storage che contiene il filesystem che il kernel monterà come root filesystem (*rootfs*, quello che parte dalla directory `/`)

Avvio del SO da parte del kernel

- L'intero SO poi partirà perché
 - Al termine del proprio avvio, il kernel Linux avvierà un eseguibile di inizializzazione dello dello spazio utente
 - Tipicamente `/sbin/init`
 - Vedremo questi passi in dettaglio, direttamente con il codice, in una lezione futura
 - Tale eseguibile si occupa tipicamente di avviare l'intero sistema operativo, ossia
 - servizi
 - interfaccia utente

systemd

- Controlliamo il file `/sbin/init`
- Per esempio con
`ls -l`
- E' molto probabile che scopriremo che `/sbin/init` è un link a *systemd*
 - Suite di componenti per l'avvio di un SO, e per la gestione di servizi e risorse in modo parallelo e efficiente
 - Ormai standard in molte distribuzioni Linux

Fare stampare messaggi all'avvio

- Finalmente tocca a noi ...
- Mettiamo assieme tutte le conoscenze acquisite per far avviare il nostro kernel in modalità cosiddetta verbose, in cui stampa effettivamente tutti i messaggi che vengono generati all'avvio

Soluzione

- Andare in edit per kernel che vogliamo avviare, eliminare `quiet` e `splash` dalla riga di comando, avviare il kernel con *Ctrl-X*

dmesg

- I messaggi scorrono via durante l'avvio, per poi sparire definitivamente
- Come visualizzarli anche durante il funzionamento del SO?
- Per esempio, con il comando `dmesg`, che scrive su *stdout* tutti i messaggi generati a partire dall'avvio del sistema
- Per attivare direttamente un visualizzatore dell'output (tipicamente `less`):

```
dmesg -H
```

Kernel log

- I messaggi del kernel sono memorizzati in un array circolare di messaggi, chiamato *kernel log*
- Quindi le istruzioni che scrivono un messaggio, all'interno del codice del kernel, aggiungono semplicemente un messaggio in fondo all'array
- Ma se l'array si riempie, il messaggio successivo sarà memorizzato al posto del primo messaggio, e così via
- Quindi il comando `dmesg` mostra il contenuto corrente del kernel log

Kernel log e fasi di avvio

- Usando il comando `dmesg`, analizziamo il kernel log
- Possiamo individuare due parti
 - Avvio del kernel: inizializzazione di tutte le risorse della macchina e di tutte le strutture dati che servono al kernel per funzionare
 - Avvio del resto del sistema operativo: avvio di tutti i servizi e della interfaccia utente
 - I componenti avviati nella seconda parte vivono in *userspace*

Livelli di criticità

- Ciascun messaggio nel kernel log è anche etichettato con un livello di criticità
- Cercate la descrizione dei livelli in

`man dmesg`

- Ottenete la lista dei livelli con

`dmesg -help`

- Esercizio: controlliamo i messaggi nel kernel log, per ciascuno dei livelli

Soluzione

```
dmesg -H -l debug  
dmesg -H -l info  
dmesg -H -l notice  
...  
dmesg -H -l emerg
```

Per chiudere sull'avvio

- Trovate tanti dettagli in questa presentazione (non da studiare per questo corso)
<https://www.slideshare.net/slideshow/vmlinux-anatomy-of-bz-image-and-how-x86-64-processor-is-booted/249346937>

Esercizio

- Dato il primo messaggio in cui il kernel scrive la propria versione, trovate, nel codice del kernel, la riga che stampa tale messaggio
- Aiuto: tale messaggio è tipicamente chiamato banner

Soluzione

<https://elixir.bootlin.com/linux/v6.16.8/source/init/main.c#L919>

Indice

- Facciamo visita al kernel ed alla sua documentazione online
- Dove ed in che forma vive il nostro kernel running, all'interno del filesystem
- Il processo di avvio
- Elixir e la nostra prima visita al codice sorgente: il codice di avvio del kernel
- I nostri primi due hack del kernel!

Il nostro primo hack del kernel

- Salutiamo il mondo da dentro il kernel!
- Facciamo scrivere un messaggio di saluto nel kernel log, prima del primo messaggio scritto attualmente dal kernel
- E' necessaria la ricompilazione e reinstallazione del kernel?
- Se sì, che comandi dobbiamo utilizzare?

Risposte

- Si
- make
 - sudo make install
 - sudo reboot

Aggiunta versione del kernel

- Come vedremo in dettaglio
 - Uno dei prodotti principali della compilazione del kernel è l'immagine del kernel
 - Tale immagine ha un suffisso: una stringa che rappresenta la versione del kernel
 - Questa è la stessa stringa che appare se si utilizza il comando `uname -r`

Domanda

- A seguito della ricompilazione effettuata nell'ultimo esercizio, al nome dell'immagine del kernel è stato aggiunto un ulteriore suffisso?

Risposta

- Se questa è stata la vostra prima modifica a questo albero dei sorgenti, allora sì
- Spiegazione nella prossima slide

Versione del kernel compilato

- Uno dei file che gestisce il meccanismo di compilazione del kernel (il top Makefile, che vedremo), contiene anche l'informazione di quale versione del kernel si sta compilando
- L'albero dei sorgenti è però anche gestito da uno strumento di controllo di versione, che si chiama `git`
 - E' `git` che conosce la versione precisa del kernel, in base a tutti i cambiamenti del codice che sono stati registrati nell'arco del tempo
- Quando si compila il kernel, il sistema di compilazione confronta la versione dichiarata nel Makefile con quella precisa, mantenuta attraverso `git`

Suffissi versione kernel

- Se le due versioni coincidono, allora al nome dell'immagine del kernel è aggiunta appunto solo la stringa che rappresenta tale versione
- Se non coincidono, ma almeno il codice che si è compilato non contiene nessuna modifica rispetto alla versione precisa del kernel (quella mantenuta attraverso `git`)
 - Allora alla stringa di versione di sopra viene aggiunta una ulteriore stringa, che identifica l'ultimo cambiamento registrato per il kernel che si sta compilando
 - Vedremo dettagli in seguito
- Se invece ci sono anche modifiche rispetto all'ultima versione precisa del kernel
 - Allora viene aggiunto un ulteriore, ultimo suffisso: `-dirty`

Il nostro secondo hack

- Come avevamo detto, alla fine del suo avvio, il kernel lancia, di default, l'eseguibile `/sbin/init`
 - Redirottiamo il boot!
 - Su un nostro eseguibile
 - Due passaggi
 - 1)Preparare l'eseguibile
 - 2)Redirottare il boot sul nostro eseguibile come init
- AVVISO!!!!!! Leggere gli avvisi nelle prossime due slide

AVVISO!!!!!!!!!!!!

- Per effettuare il reindirizzamento, non modificate in alcun modo alcun file nel filesystem
- Per esempio
 - Non modificate il link `/sbin/init`
 - Non modificate `/etc/default/grub`
- Qualsiasi modifica al filesystem è permanente ed indipendente da quale kernel avviate
 - Quindi il vostro sistema non avvierà più l'ambiente userspace

Secondo avviso

- Come avrete probabilmente già capito dalla piccola esperienza che già state facendo, un kernel custom con configurazione minimale è statisticamente più instabile di un kernel stock con configurazione ottimizzata dagli sviluppatori della distribuzione in cui gira
- Quindi, per questo hack, provate prima il reindirizzamento dell'avvio di un kernel stock
 - Se funziona, provate poi anche con il vostro kernel custom

Il nostro secondo hack: suggerimenti

- Per il primo passaggio, va benissimo un programma che stampi il classico *ciao mondo*
 - Domanda in merito, se questo programma viene effettivamente lanciato come init, che succederà se/quando tale programma terminerà?
- Per il secondo passaggio, vi ricordo la documentazione: <https://docs.kernel.org/admin-guide/kernel-parameters.html>
 - Trascurare il parametro `rdinit`
 - Ne ripareremo in una prossima lezione

Domanda

- Il file di hello world che volete far eseguire deve essere un eseguibile?
- Deve essere collegato staticamente?

Risposta

- Sì, deve essere un eseguibile
- No, non è necessario che sia collegato staticamente
- Perché dopo l'avvio del kernel è già funzionante il collegamento dinamico all'atto del caricamento degli eseguibili

Primo esempio di programma

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

- E' ok? Che problema ha?

Versione che rimane in vita

```
#include <unistd.h>
#include <stdio.h>

int main(){
    printf("Hello, world!\n");
    sleep(10);
    return 0;
}
```

Redirottamento dell'avvio

- Andando in edit sulla entry del kernel che si vuole avviare, aggiungere

```
init=<percorso_nostro_programma>
```

alla riga di comando, in qualsiasi posizione dopo il nome dell'immagine del kernel

Esercizio più difficile

- Opzionale
- Non banale
- Estendere il precedente programma affinché, dopo aver scritto il proprio messaggio e atteso, faccia proseguire l'avvio del sistema operativo
- C'è un aiuto nella penultima slide, dopo la slide di chiusura di questa parte del corso
 - Leggerlo solo quando si è sicuri di non aver nessuna idea
 - Nell'ultima slide c'è poi un esempio di soluzione

Fine della prima parte del corso di
Kernel Hacking

Domande?

Aiuto

```
man exec.3
```

Esempio di soluzione

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Hello World!\n");
    char sbin[] = "/sbin/init";
    char *argv[] = {"init", NULL};
    char *envp[] = {"HOME=/", "TERM=linux", NULL};

    execve(sbin, argv, envp);
    return 0;
}
```